

Embedded Systems and IoT

Ingegneria e Scienze Informatiche - UNIBO

a.y. 2025/2026

Lecturer: Prof. Alessandro Ricci

[module 1.1] INTRODUCTION TO EMBEDDED SYSTEMS

SUMMARY

- This module provides an overview of main concepts about embedded systems
 - basic terminology and concepts
 - embedded system architecture
 - intro to design and development of embedded software

EMBEDDED SYSTEMS: DEFINITION

- **Special-purpose** computing systems
 - i.e. they do a specific function or task *embedded* in devices and physical/electronic systems, of different sizes and with different purposes
 - the task of the embedded system requires an interaction with the physical world by means of **sensors** and **actuators**
- Typically composed by two parts: hardware + software
 - sometime the sw part can be missing
- **Embedded software**
 - software part of an embedded system

EMBEDDED SYSTEM PERVASIVENESS

- Consumer electronics
 - smartphones, cameras, wearables,...
- Home appliances
 - thermostats, smart lighting systems, smart fridges, robot vacuum cleaners, alarm systems, ...
- Office automation
 - copy machines, printers,...
- Business equipment
 - ATM, cash register, ...
- Cars, vehicles, ...
 - drive system, navigation system, ...
- ...

HOME APPLIANCES



INTERNET OF THINGS e SMART THINGS



Nest smart thermostat

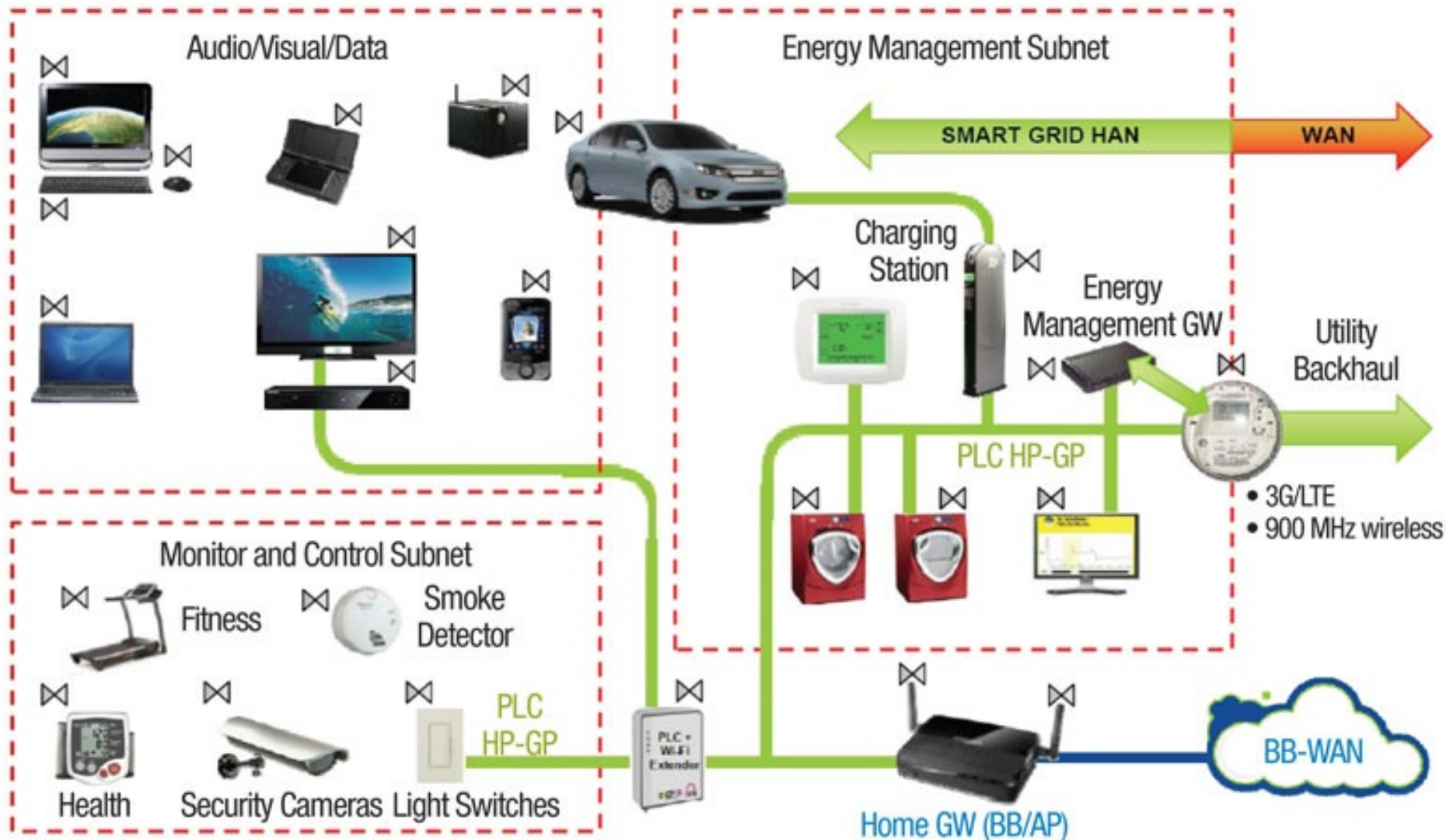


Kevo smart lock

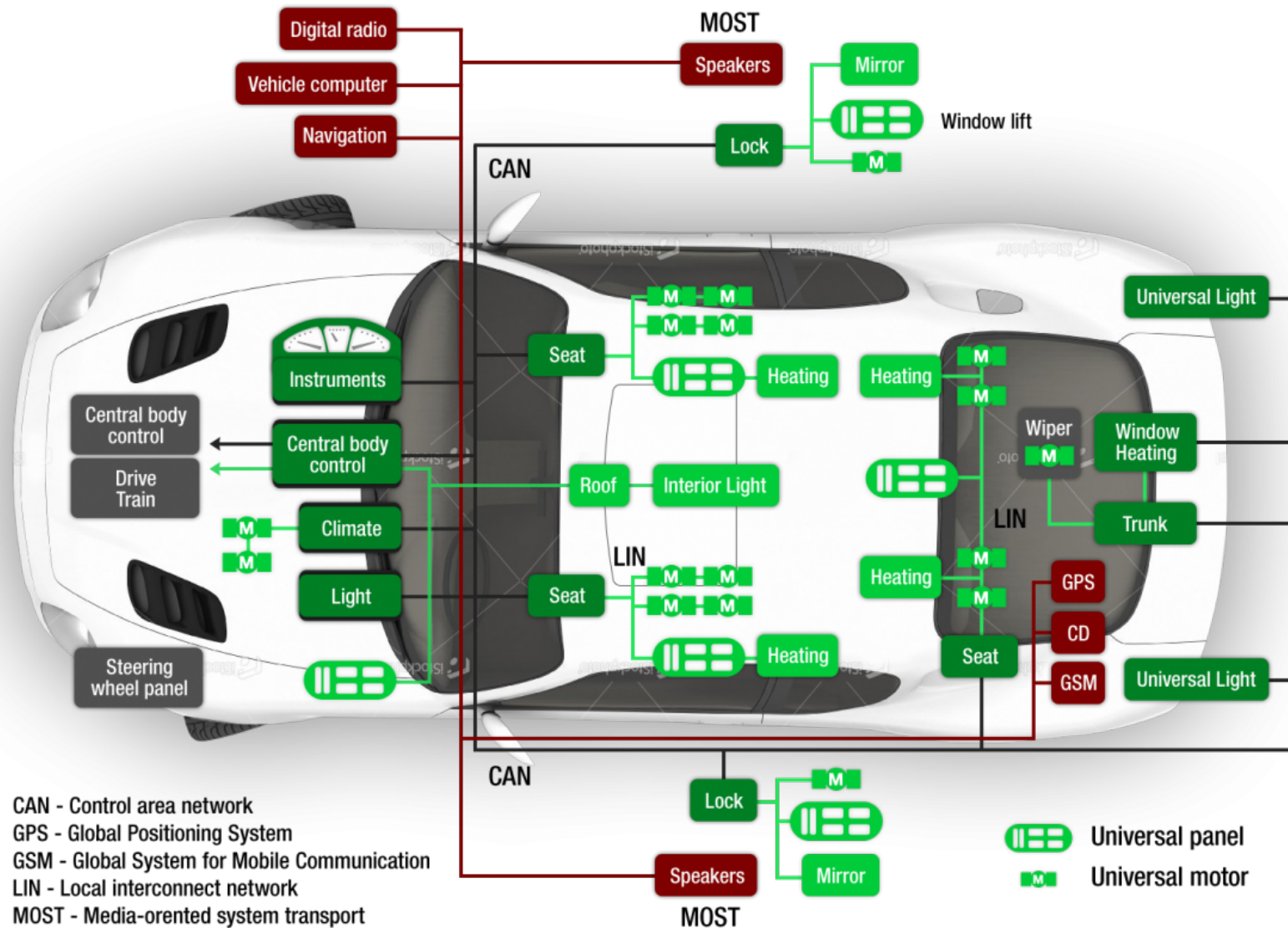


WeMo smart things

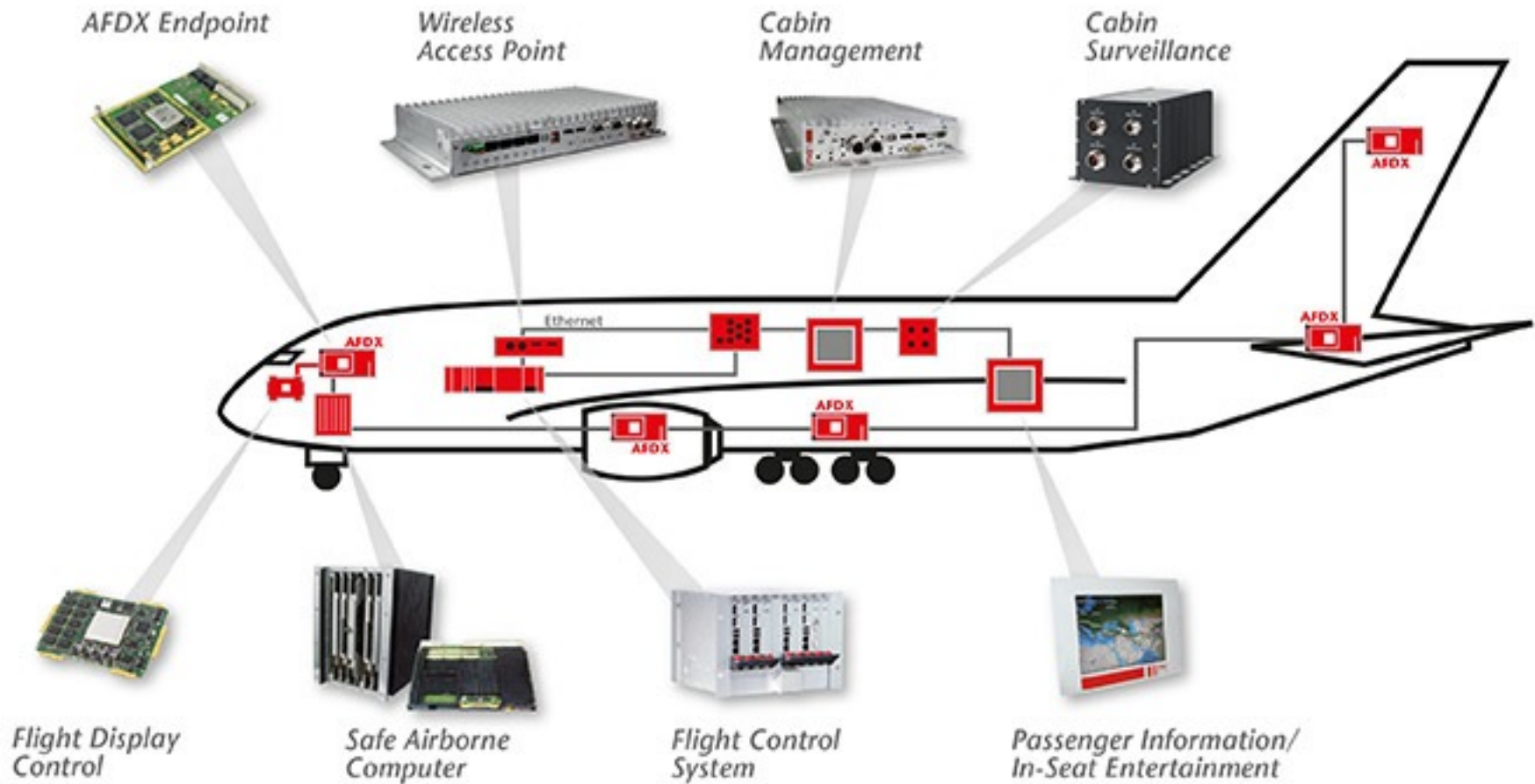
SMART HOME



AUTOMOTIVE

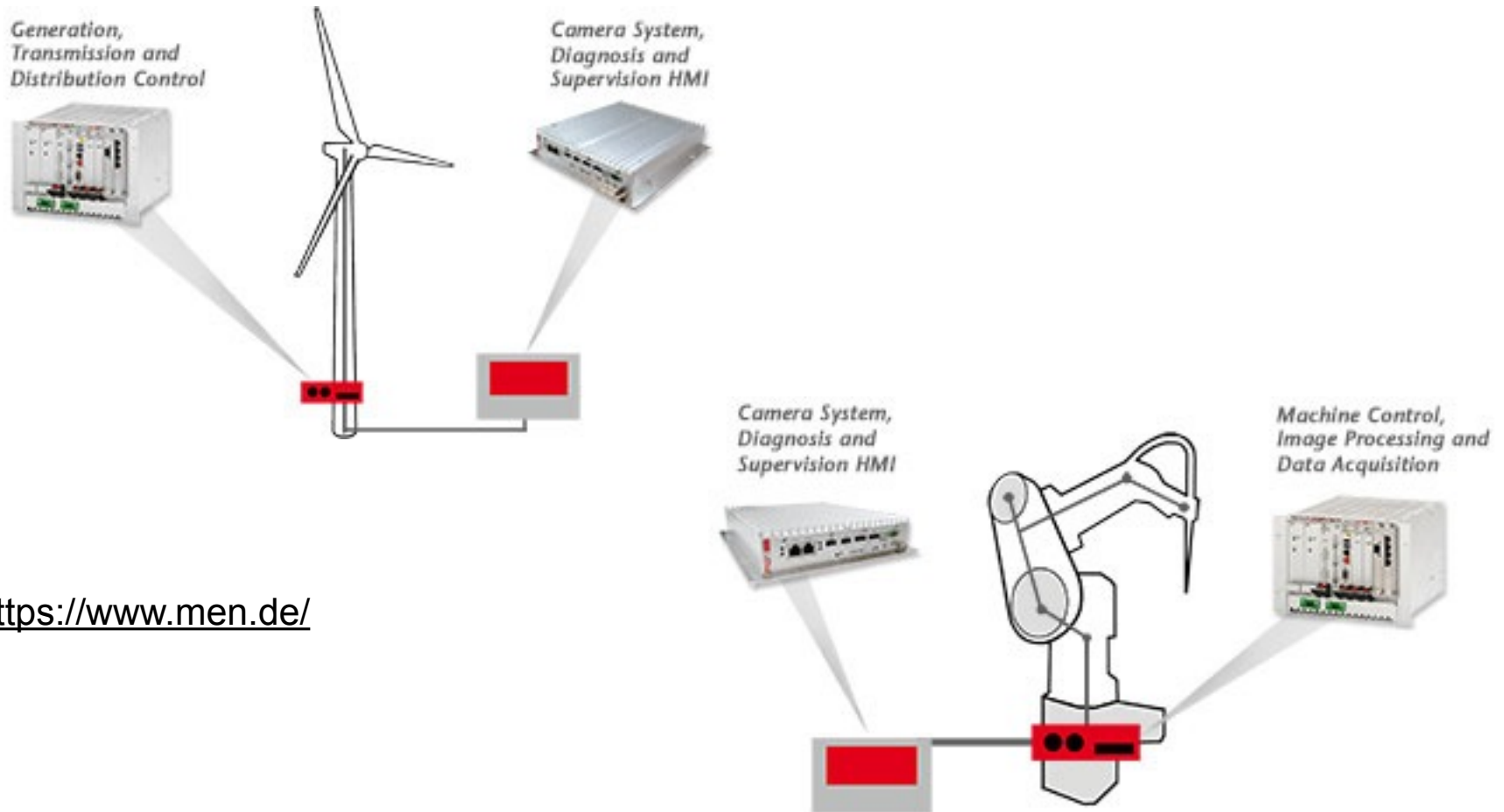


AVIONICS



“Airborne functions in commercial aircraft are controlled by modular avionics systems. Embedded computers take on vital functions such as flight and engine control, or navigation.”
(<https://www.men.de/>)

INDUSTRY

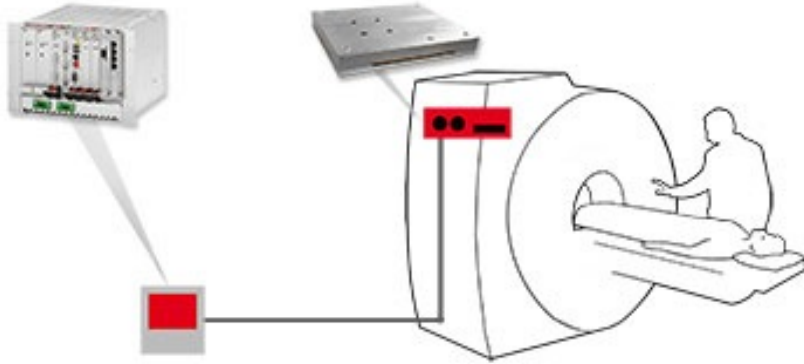


<https://www.men.de/>

HEALTHCARE

*Diagnosis, Monitoring and
Patient Communication Systems*

*Surgical Control, Processing
and Acquisition Components*



WristOx

MOBILE E WEARABLE DEVICES



smartglasses



smart-watch

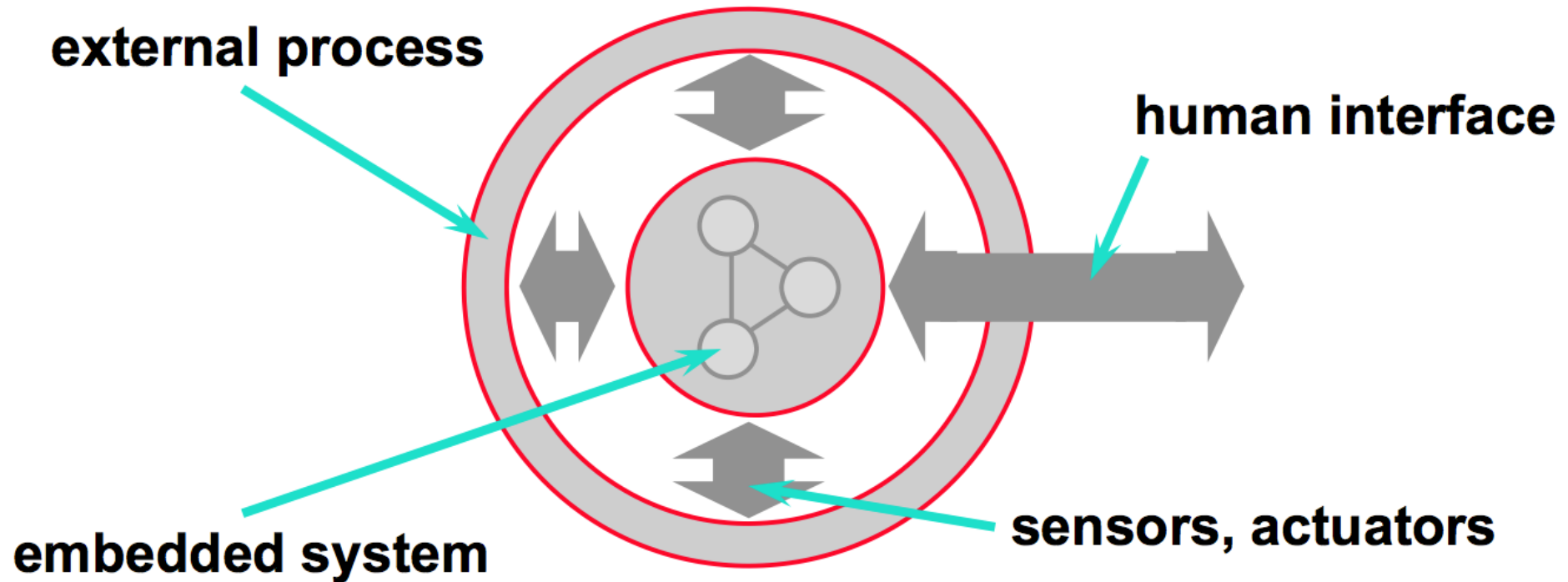


Fitbit



smart armband

EMBEDDED SYSTEM *EMBEDDING*



(Embedded system introduction, http://www.tik.ee.ethz.ch/education/lectures/ES/slides/1_introduction)

CHARACTERISTICS

- **Specific purpose/functionality**
 - an embedded system typically executes one specific application
 - never-ending program (a process)
 - typically executing a task *repeatedly*
 - designed for robustness and minimisation of used resources
 - ad hoc user interface and experience

CHARACTERISTICS

- **Tightly constrained (resource) ed efficiency**
 - design constrains typically more strict in terms of available CPU/memory/power/... than desktop/server systems
- Use of *design metrics*
 - measuring characteristics of implementation/development
 - **cost** - including **NRE** costs (non-recurring-engineering = one time cost)
 - **size, performance, energy consumption**
- Design oriented to **efficiency**
 - energy efficient, code-size efficient, run-time efficient, weight efficient, cost efficient

CHARACTERISTICS

- **Critical systems**
 - embedded systems often play a critical role in the context where they are used
 - e.g. healthcare devices, transportation, industry..
- This calls for a design oriented to
 - **dependability, reliability, availability**
 - high probability of proper and continuous functioning
 - **safety and security**
 - never arming users or environments
 - strict security and privacy norms

CHARACTERISTICS

- **Reactivity** and **real-time**
 - embedded systems are typically used in application contexts in which the system must *promptly* react to stimula from the physical environment, by performing computations and actions in real-time, *within a deadline*, without latencies
 - in the case of *hard real-time*, the deadline violation is considered a failure of the system and could cause critical problems

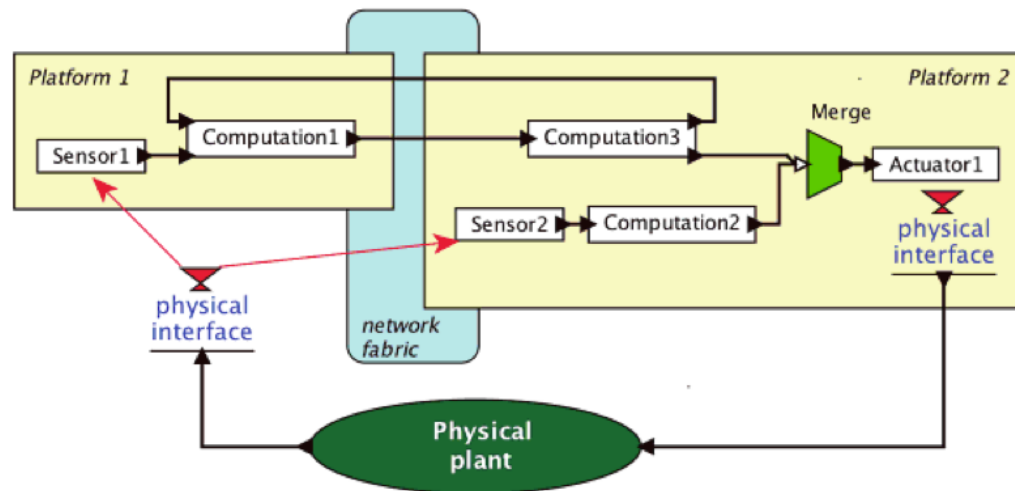
CYBER-PHYSICAL SYSTEMS (CPS)

- **Cyber-Physical Systems (CPS)**
 - systems that *integrates computations with physical processes*
 - vs. information processing systems
- Specific characterising aspects compared to information processing systems:
 - strong **time dependency** and management of time
 - not only for performance, but correctness
 - **concurrency**
 - physical processes are typically in parallel
 - **reactivity, asynchronous events**
 - reactive systems vs. transformational systems

CYBER-PHYSICAL SYSTEMS (CPS)

- 3 parts or subsystems
 - **physical part**
 - can include devices, mechanical/biological systems, chemical processes, human operators (users)
 - **computational/embedded part**
 - one or multiple computing platforms, each equipped with sensors, actuators, and one or multiple computers
 - **network part**
 - provides the mechanisms to connect the computing platforms

cyber {

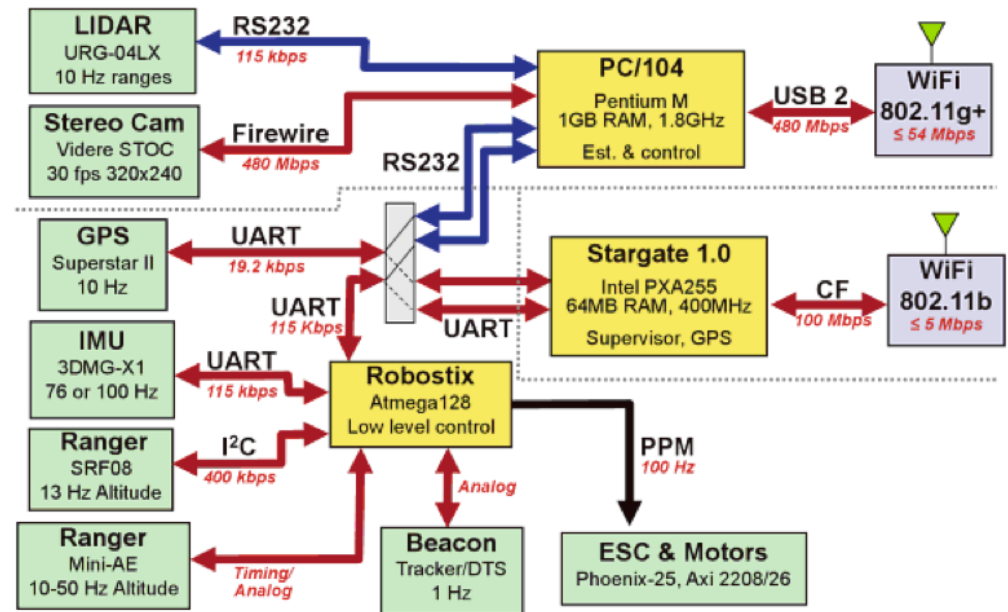


(IES book, p. 6)

EXAMPLE: DRONES



(IES book, p. 7)



CPS: INTER-DISCIPLINARITY

Software

```

// Read the whole molecules, which are the latest molecules of
// the output part of the partition.
// Message::TtlGetMolecules() of getting the molecules, for
// A1.
private void _receiveMolecules() throws TtlGetMoleculesException {
    TtlGetMoleculesResponse r = (TtlGetMoleculesResponse) getMolecules();
    for (TtlGetMoleculesResponse r : getMolecules()) {
        if (_isMolecule()) {
            _debug("Receiving latest molecules of output part: "
                + output.getPartId());
        }
        Receiver[] receivers = output.getReceiverMolecules();
        if (receivers != null) {
            for (int i = 0; i < receivers.length; i++) {
                if (receivers[i] != null) {
                    for (int j = 0; j < receivers[i].length; j++) {
                        if (receivers[i][j] != null) {
                            receivers[i][j].receive();
                        }
                    }
                }
            }
        }
    }
}

```

Control/
Physics

$$\dot{\mathbf{x}}(t) = \dot{\mathbf{x}}(0) + \frac{1}{M} \int_0^t \mathbf{F}(\tau) d\tau$$



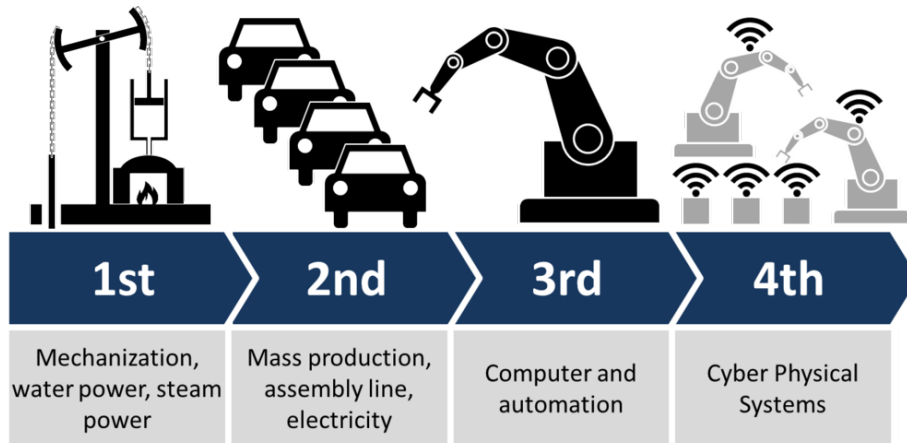
Systems



Overcome interdisciplinary boundaries

INDUSTRY 4.0

- Industry 4.0, Industrie 4.0 o fourth industrial revolution
 - affects industrial manufacturing and industry in general
- Also referred as ***smart factory***



https://en.wikipedia.org/wiki/Industry_4.0

DIGITAL TRANSFORMATION OF FACTORIES

- It includes **cyber-physical systems**, **Internet of Things (IoT)** (Industrial IoT) and **cloud computing**
 - ***cyber-physical systems*** monitor physical processes, creating a virtual representation of physical assets, allowing for remote/distributed decision making
 - By exploiting ***IoT***, cyber-physical systems communicate and cooperate through the Internet, and with human operators in real time
 - ***cloud-based*** Internet services are made available both inside an organisation and outside for partners that are part of the value chain

“PHYSICAL COMPUTING” PERSPECTIVE

- Terms used to refer computational systems that interact with the physical world, typically targeted to the construction and exploration of human-machine interaction frameworks

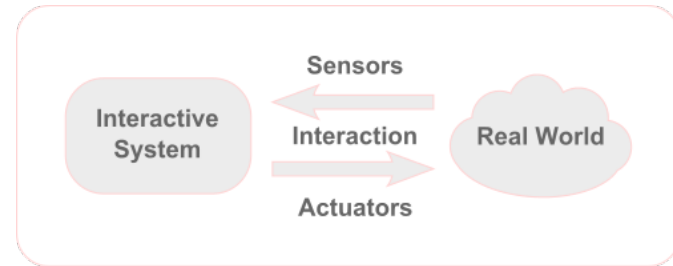
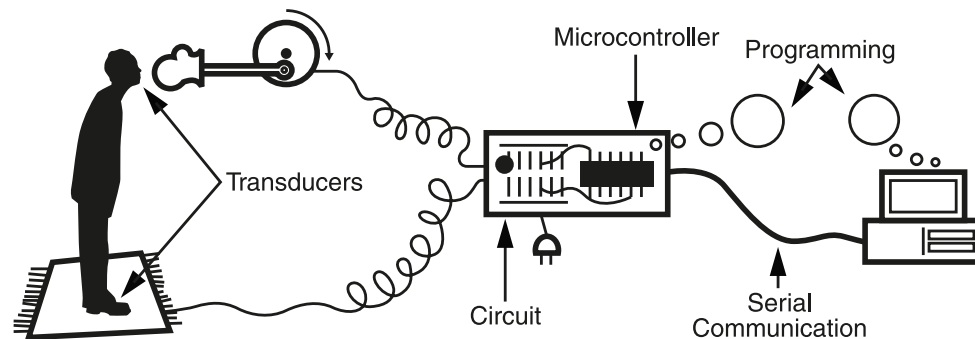


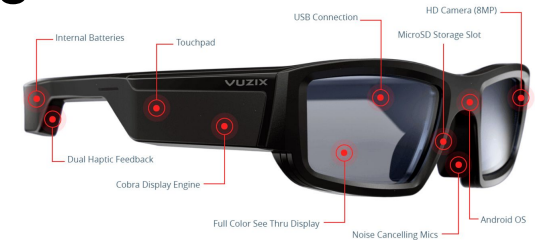
Figure 1.4
The parts of a physical computing system.



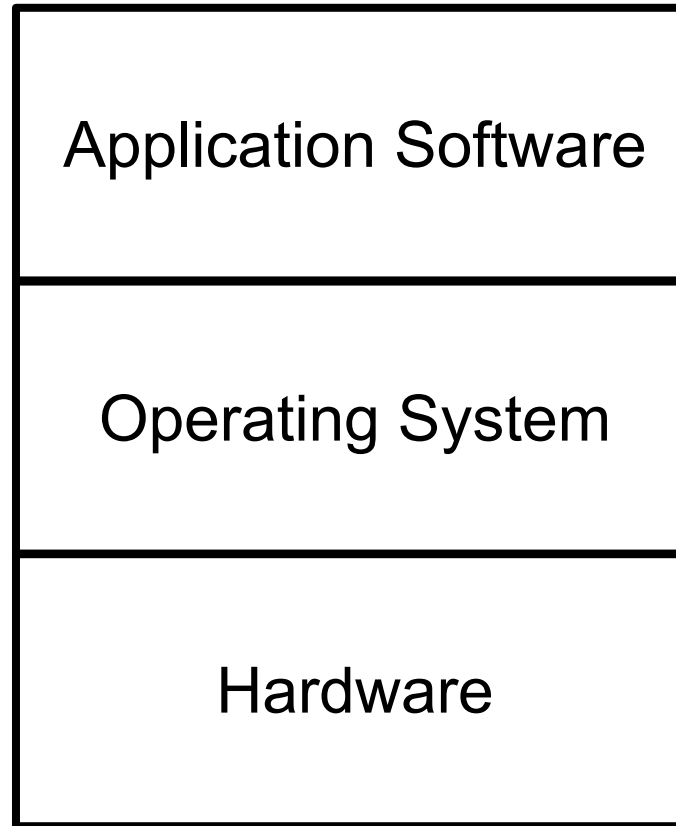
(“Physical Computing”. Dan O’Sullivan, Tom Igoe. Thomson)

WEARABLE COMPUTING

- Wearable embedded systems
 - examples
 - smart wrists, smart-watches, smartglasses
 - eyewear computing
 - based on smartglasses
 - spatial computing
 - based on mixed/augmented reality systems
- Enabling technologies for
 - hands-free systems

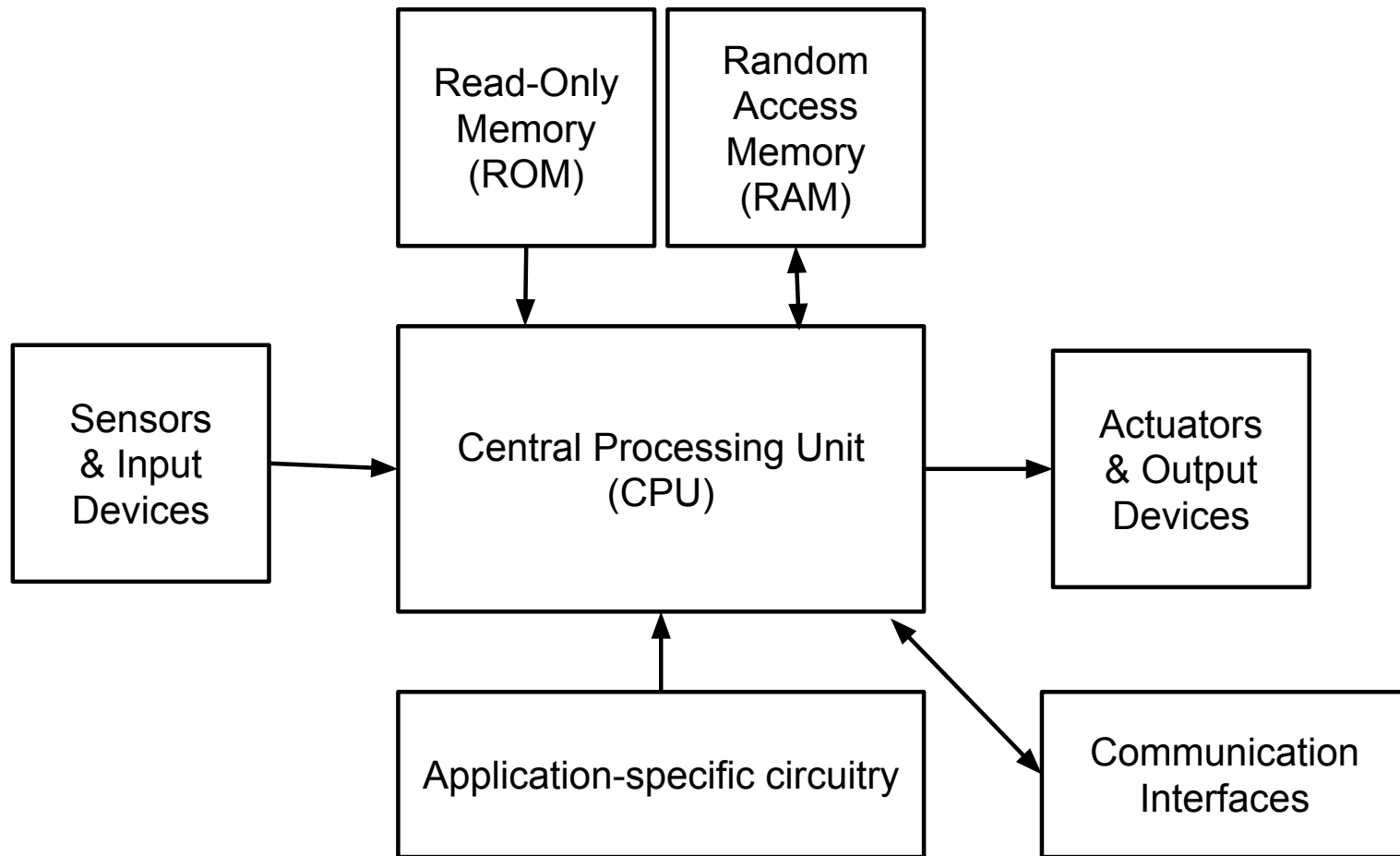


EMBEDDED SYSTEM ARCHITECTURE

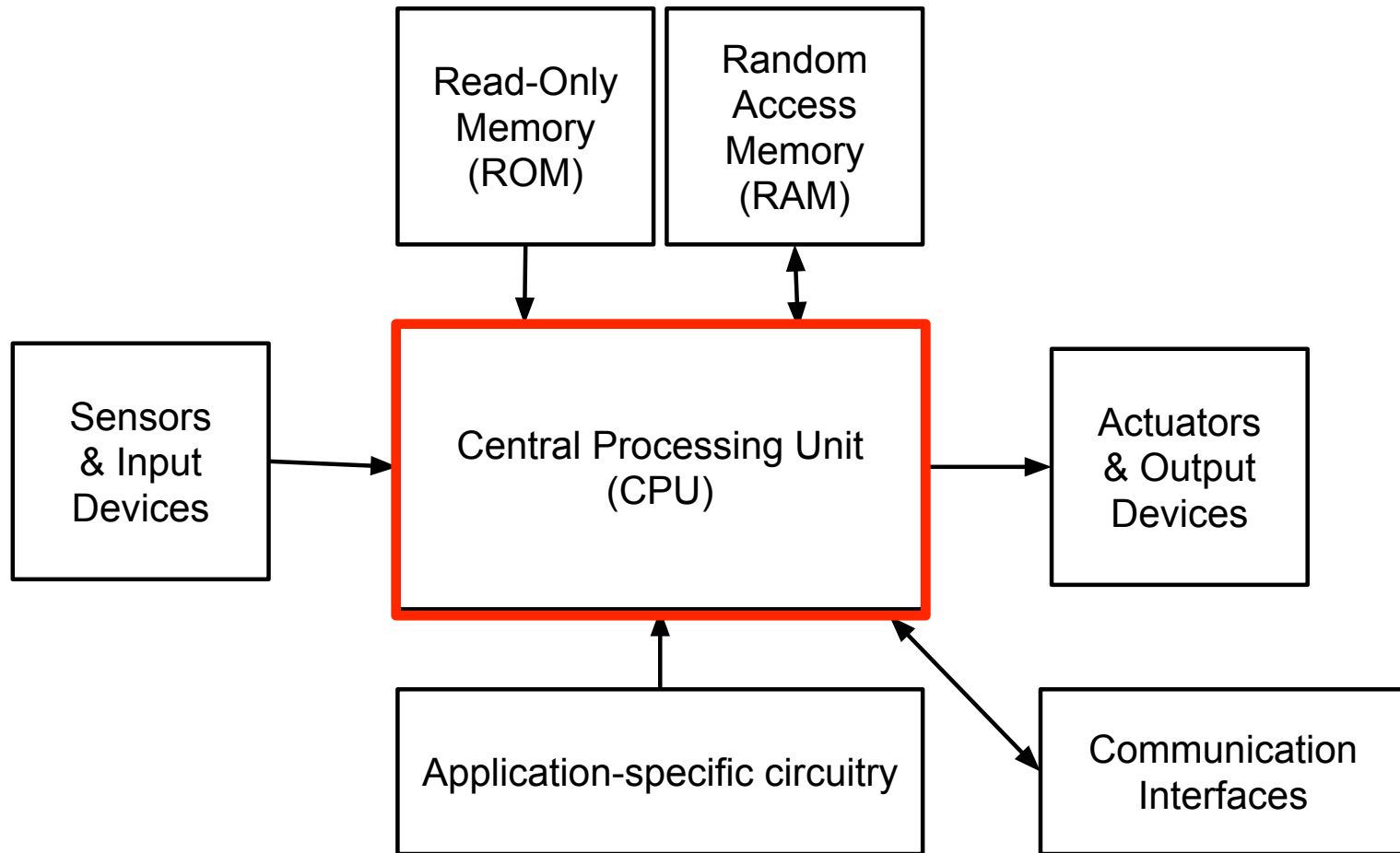


(the software layers could be missing)

HARDWARE ARCHITECTURE



HARDWARE ARCHITECTURE: CPU



CPU: TECHNOLOGIES

- Three kinds of processor technologies
 - **general-purpose processors** ($\Rightarrow$ *software*)
 - processors featuring a predefined architecture and an instruction set (instruction set architecture, ISA) e the behaviour of the system is defined by the **program (software)** in execution
 - **single-purpose processors**
 - digital circuits designed to implement a specific function or program
 - **ASIC** = Application-Specific Integrated Circuit
 - **Application-Specific Processors (ASIP)**
 - middle case
 - programmable processors, optimised for a specific class of applications with common features
 - e.g.: **micro-controller units or microcontrollers (MCU), SoC (System-on-a-Chip)**, DSP (Digital Signal Processors)
 - even in this case, the application logic is defined at the software level

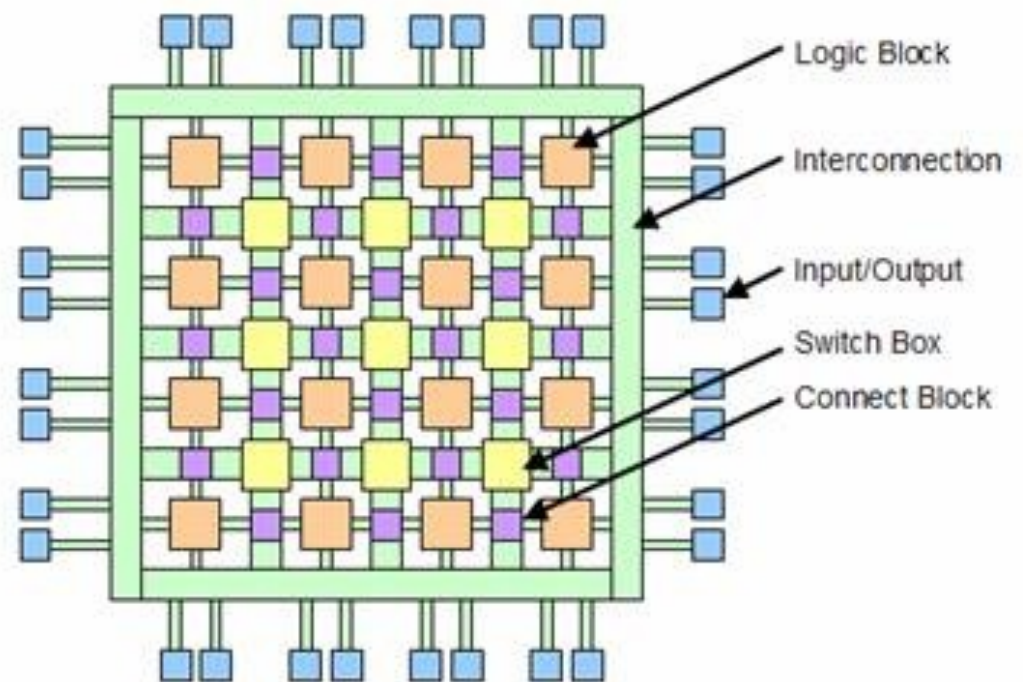
SINGLE-PURPOSE PROCESSORS

- Different implementations
 - **Full-Custom/VLSI**
 - the implementation accounts for designing the full processor in a custom way, optimising all levels
 - **Semi-custom**
 - **ASIC** - application specific IC - in this case the starting point is a set of predefined levels, implementing the processor on top
 - **PLD - programmable logic device**
 - all levels are already there, but can be configured/“programmed” for the purpose
 - the levels implement programmable circuits, where the programmability is given by the possibility of create/destroy connections, gates, etc.

SINGLE-PURPOSE PROCESSORS

- PLA (Programmable Logic Array)
 - programmable array of AND gates + OR gates
- PAL (Programmable Array Logic)
 - single type of array
- **FPGA (Field Programmable Gate Array)**
 - integrated circuits with programmable functions via software
 - more general level of programmability compared to PLA and PAL
 - allows for implementing complex logics, strong scalability
 - millions of gates for a single FPGA device
 - programming languages: Verilog, **VHDL**
 - schematic-entry visual-based programming languages
 - **LabView**

FPGA



FPGA PROGRAMMING

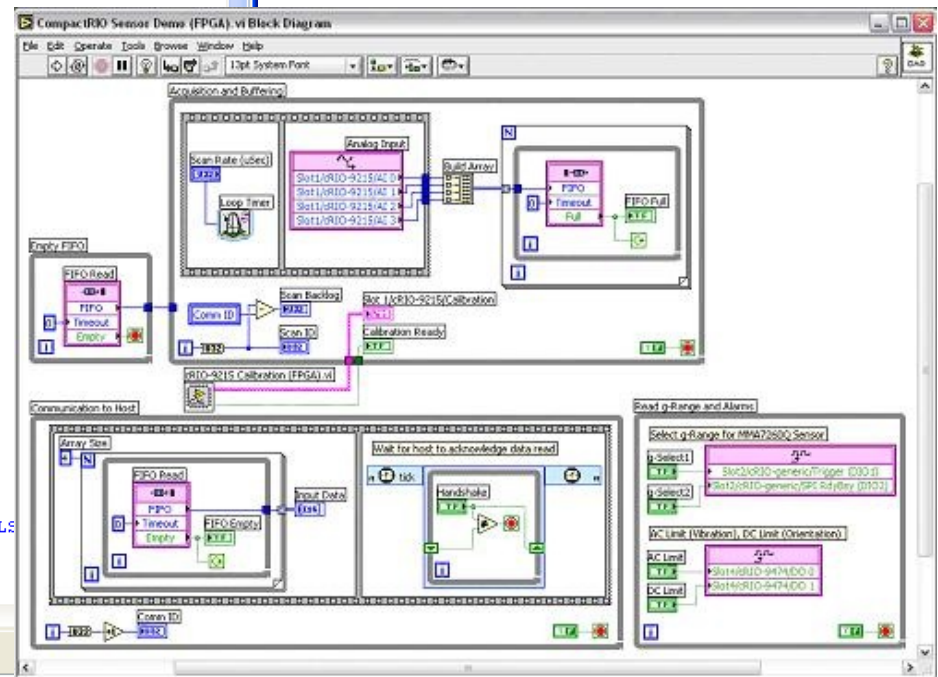
The screenshot shows the Xilinx ISE IDE with the following components:

- Sources Panel:** Lists the project files, including 'mrc-nand' and 'nand_gate - algorithmic'.
- Processes Panel:** Shows the design process steps: Add Existing Source, Create New Source, View Design Summary, Design Utilities, User Constraints, Synthesize -XST, Implement Design, and Generate Programming File.
- VHDL Code:**

```

30
31 Library IEEE;
32 use IEEE.std_logic_1164.all;
33
34
35 ENTITY nand_gate IS
36   port
37   (
38     a,b : in  std_logic;
39     c   : out std_logic
40   );
41 END nand_gate;
42
43
44 ARCHITECTURE dataflow OF nand_gate IS
45
46 BEGIN -- dataflow
47   c <= not ( a and b ) ;
48 END dataflow;
49
50
51 ARCHITECTURE dataflow_delay OF nand_gate IS
52
53 BEGIN -- dataflow_delay
54   c <= not ( a and b ) after 10 ns ;
55 END dataflow_delay;
56
57
58 ARCHITECTURE algorithmic OF nand_gate IS
59
60 BEGIN -- algorithmic
61   PROCESS (a,b)
62   BEGIN -- PROCESS
63     IF ( ( a='1' ) and ( b='1' ) ) THEN c<='0'; ELSE
64     END PROCESS;
65 END algorithmic;
66

```
- Transcript Panel:** Shows the message "Started : 'Launching Design Summary'".

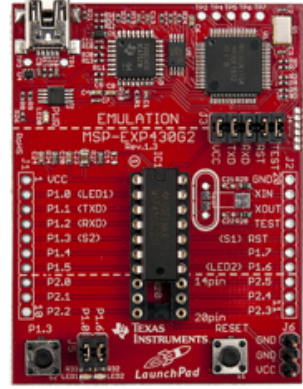


MICROCONTROLLER UNIT (MCU)

- Differently from traditional microprocessors, **microcontrollers integrate on a single chip all the components required to have full functional autonomy for the tasks that the embedded is designed for**
 - CPU, persistent memory, volatile memory, I/O channels (pins), interrupt controller unit, and other specialised components
 - both CISC CPU based on Von Neumann architecture, and RISC based, such as the Texas Instruments MSP430
 - 8, 16, 32 bit
- Examples
 - historical: 8 bit MCU: Motorola 68000, Intel 8080 (1975), Zilog Z80, Intel 8051
 - more recent: Atmel AVR, Texas Instruments MSP430, Microchip PIC16C84, ARM a 32 bit, Hitachi H8, PowerPC
- Fragmented marketplace, many OEMs and many architectures, no one dominant

MICROCONTROLLERS - EXAMPLES

- List: https://en.wikipedia.org/wiki/List_of_common_microcontrollers

TI MSP430	- Well-known micro from Texas Instruments, widespread 16 bit, ultra-low-power - LaunchPad Development Kits - from 16 to a 25 MHz, from 32 KB up to a 512 KB Flash, from 2 up to 66 KB SRAM	
PIC 16505	PIC16C Microchip - 8 bit MCU CMOS - 5 MIPS, 1,5 KB memory for programs, RAM up to 72 bytes .- clock 4 MHz	
ATMega328P	8 bit MCU, but instruction takes one or two 16-bit words, 2KB SRAM, 32KB FLASH, 16 MHz	

MICROCONTROLLERS VS MICROPROCESSORS

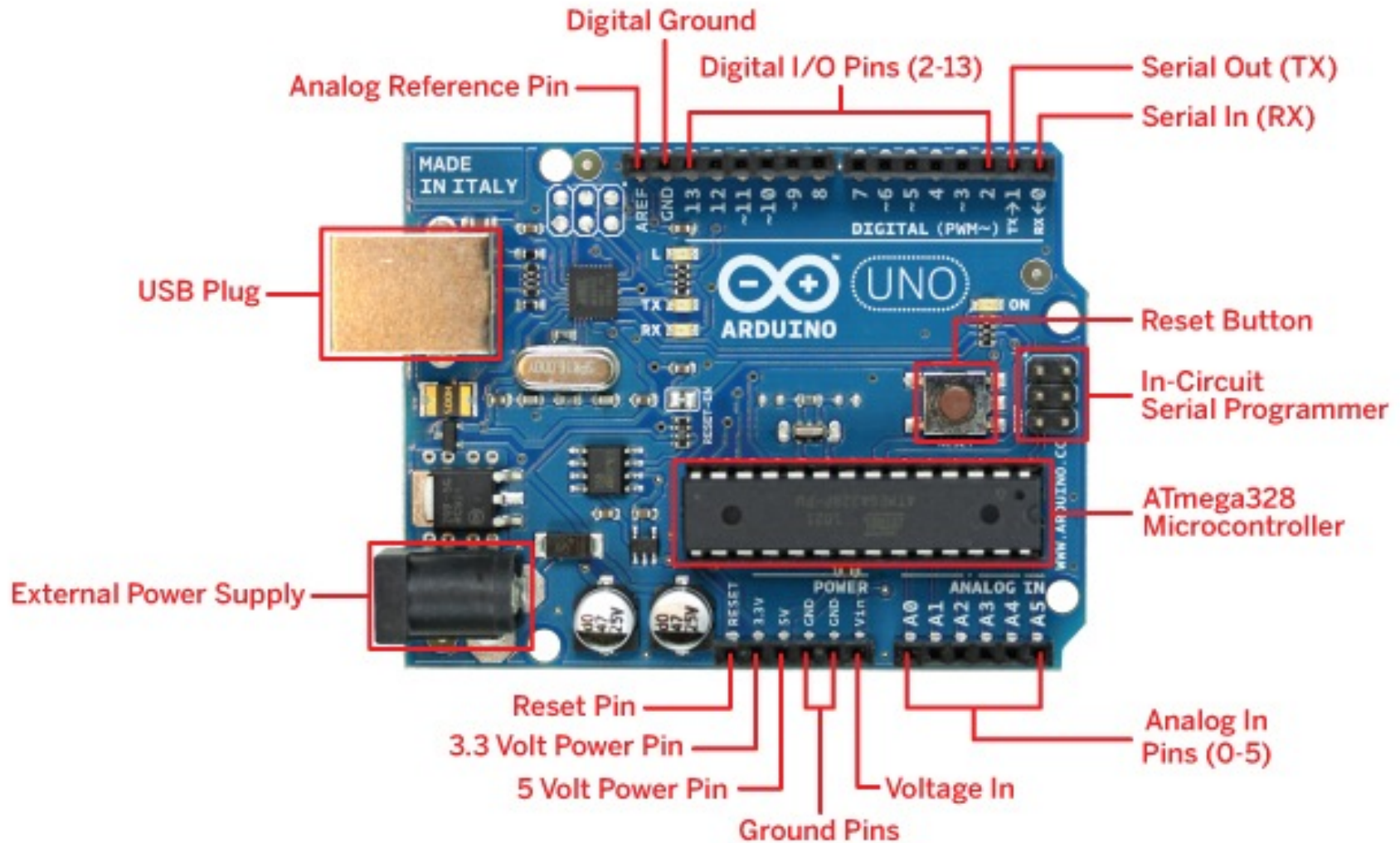
Caratteristica	Microcontroller	Microprocessor
Max clock speed	200 MhZ	4GHz
Max MegaFLOPS	200	5000
Power consumed (watt)	1	50
Minimum price per item (USD)		50
Number of items sold	11000	1000

<https://en.wikipedia.org/wiki/Microcontroller>

SINGLE-BOARD MICROCONTROLLER

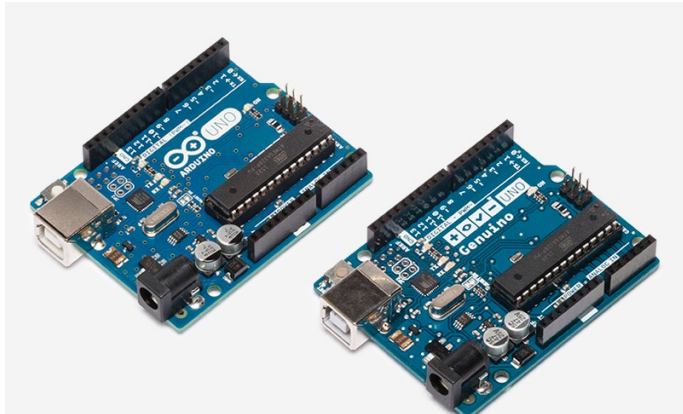
- Solutions that integrate in a single board a microcontroller and all necessary circuits and electronics to perform a control task
 - the board can be immediately used then by engineers to develop and run the embedded software, without the need to design and develop any HW for the control
- Examples:
 - consumer market/makers: Arduino
 - industries: XMC family

ARDUINO UNO (R3)

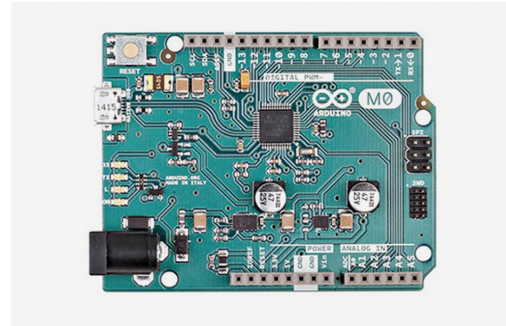


ARDUINO FAMILY

<http://arduino.cc>



Uno - 8 bit 16 Mhz, 2+32KB, 14+6 pin



M0 - 32 bit 48 Mhz, 32+256KB



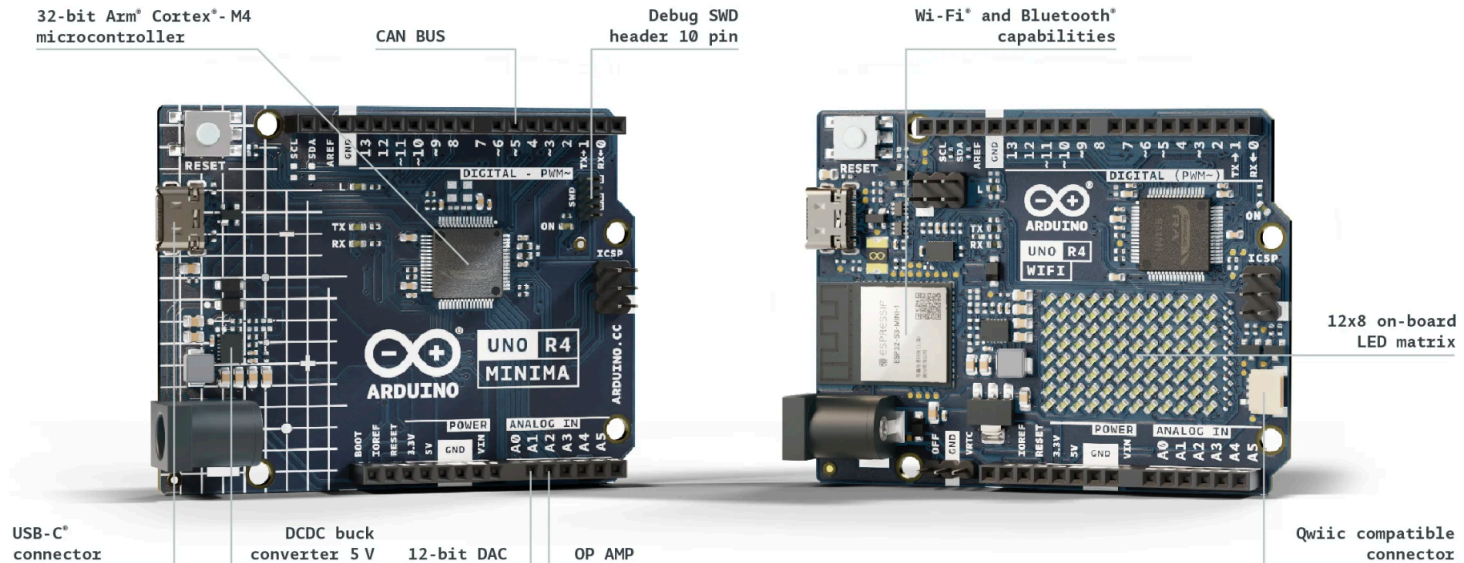
Mega - 8 bit 16 Mhz, 8+256KB, 54+16 pin



Yun - 32 bit 400 Mhz, 64+16MB



RECENT EXTENSIONS



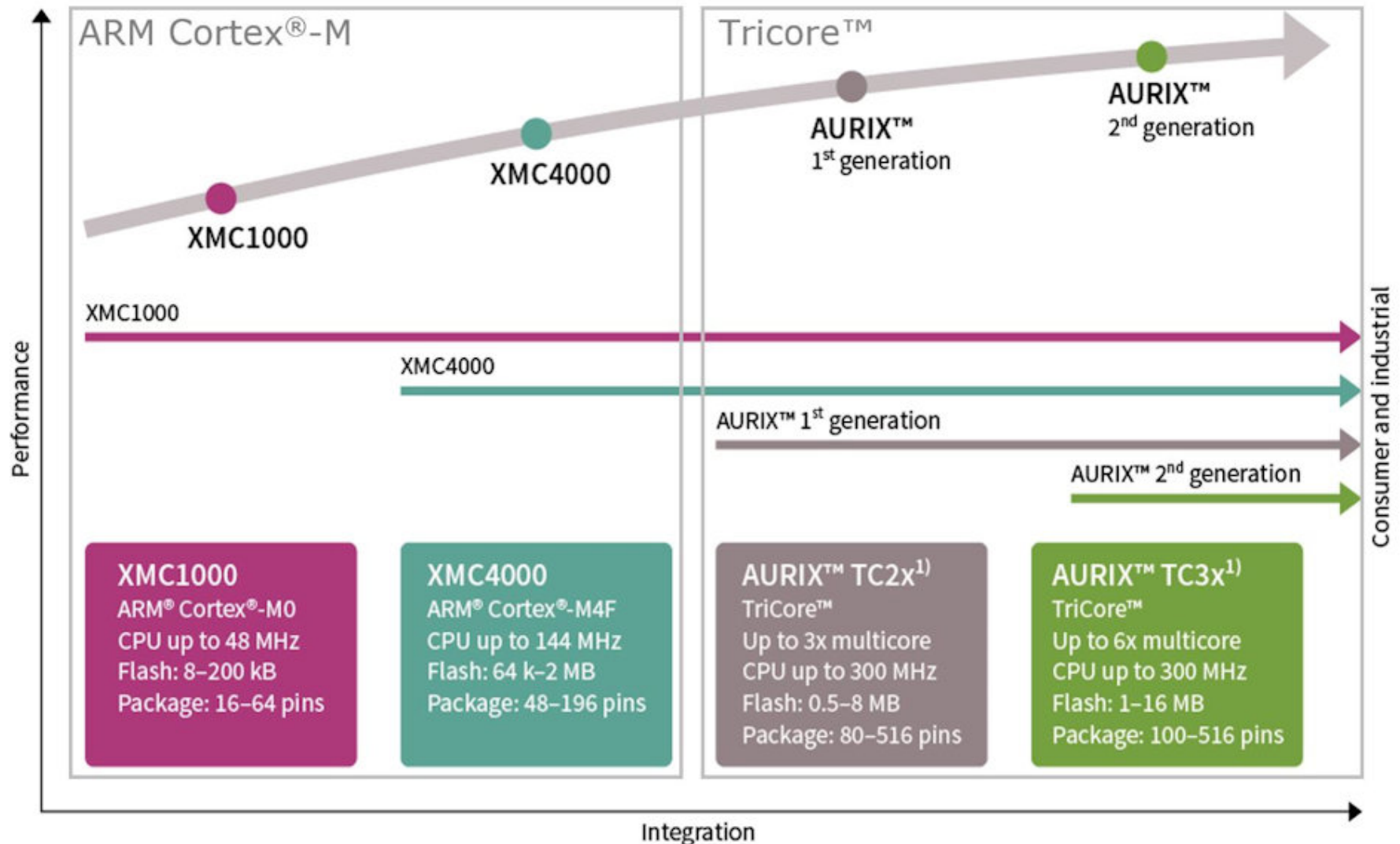
UNO R4 Minima

Focused on simplicity, it offers essential functions that make it ideal for prototyping and basic learning.

UNO R4 WiFi

Perfect for IoT and wireless projects with its ESP32-S3 for WiFi/BLE and a 12x8 onboard LED matrix for stunning visual displays.

INDUSTRIAL EXAMPLE



SOC AND SINGLE-BOARD CPU

- **SOC = System-on-a-Chip**
 - chip integrates a complete system, including CPU, memory, I/O controllers, network controllers
 - similar to microcontroller, but typically more powerful
 - 32 or 64 bit, clock ~ hundred of MHz up to GHz, memory ~ hundred of MB
 - typically used to create single-board CPU
- Esempi:
 - BROADCOM BCM2837 64bit ARMv8 Cortex A53 Quad Core (1.2Ghz)
 - used in Raspberry Pi 3
 - ARM Sitara AM335x SoC - including ARM Cortex-A8 processor
 - used in BeagleBone, Arduino Tre
 - Texas Instruments OMAP3530
 - ...

RASPBERRY FAMILY

Raspberry Pi (2012)

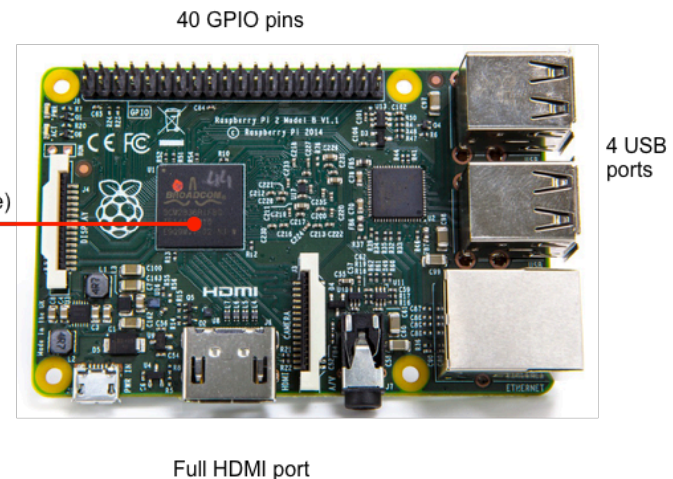
- SOC: Broadcom BCM2835
- 700 MHz single-core ARM1176JZF-S
- 256 MB



Raspberry Pi 2 (2015)

- SOC: Broadcom BCM2836
- 900 MHz 32-bit quad-core ARM Cortex-A7
- 1 GB RAM

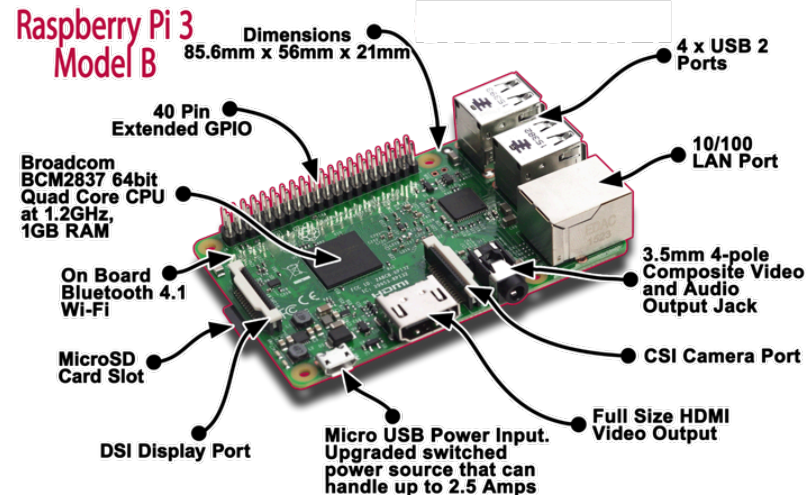
CPU:
ARM Cortex A7
(900 MHz, quad-core)
1 GiB RAM



RASPBERRY FAMILY

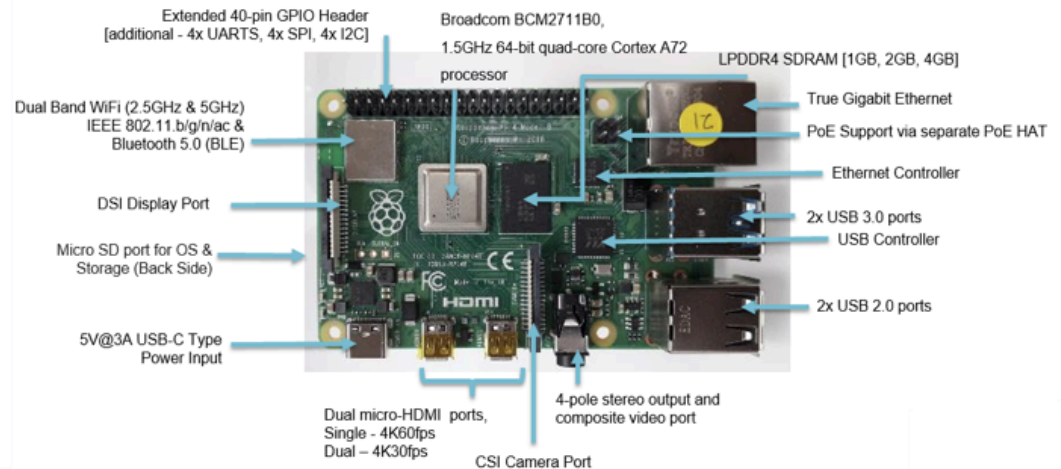
Raspberry Pi 3 (2016)

- Broadcom BCM2837
- 1.2 GHz 64-bit quad-core ARM Cortex-A53
- 1 GB RAM



Raspberry Pi 4 (2019)

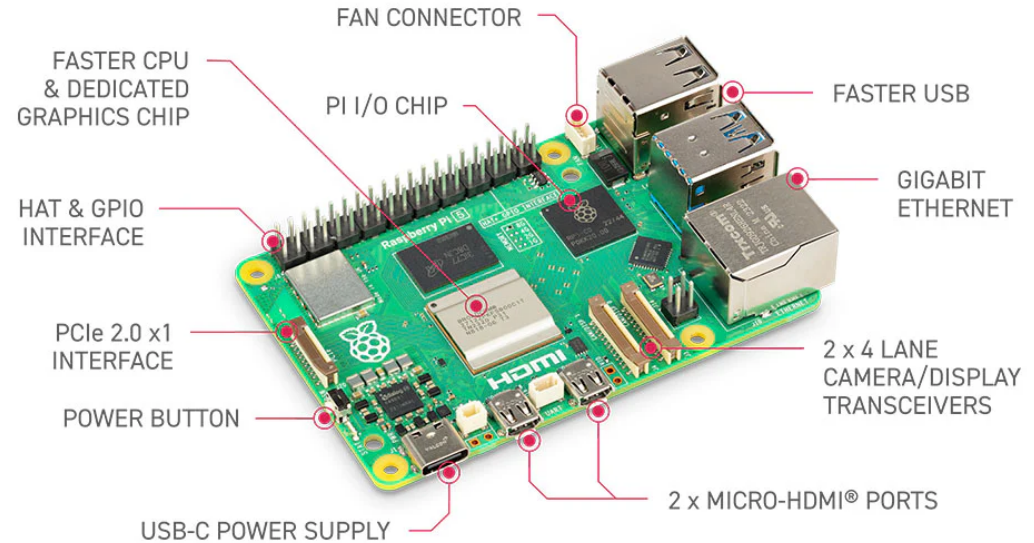
- Broadcom BCM2711
- 4x Cortex-A72 1.5 GHz
- 1, 2, 4 GB RAM



RASPBERRY FAMILY

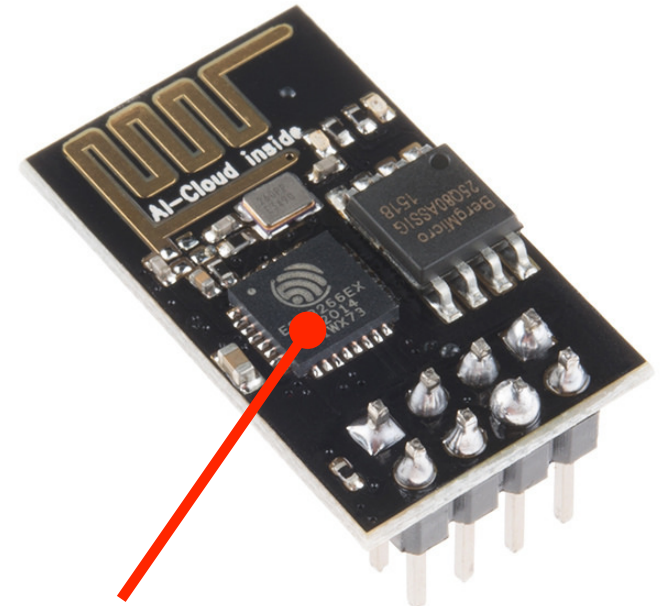
Raspberry Pi 5 (2023)

- Broadcom BCM2712
2.4GHz quad-core 64-bit Arm Cortex-A76 CPU, 512KB per-core L2 caches, and a 2MB shared L3 cache
- VideoCore VII GPU
- SDRAM: 4GB and 8GB SKUs



ESP 8266

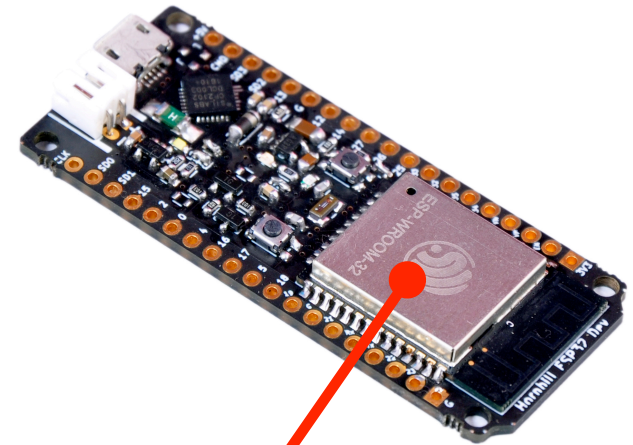
- SoC designed for IoT
- Hardware
 - Processor: L106 32-bit RISC microprocessor core 80 MHz*
 - 64 KiB of instruction RAM, 96 KiB of data RAM
 - 16 GPIO pins
 - SPI, I²C, I²S + UART
 - 10-bit ADC
 - IEEE 802.11 b/g/n Wi-Fi
- Different kind of SDK and firmware
 - GCC toolchain, Wiring / Arduino, NodeMCU Lua-based, MicroPython Python based, ESP-Open-RTOS Open source FreeRTOS-based...



**ESP8266
SoC**

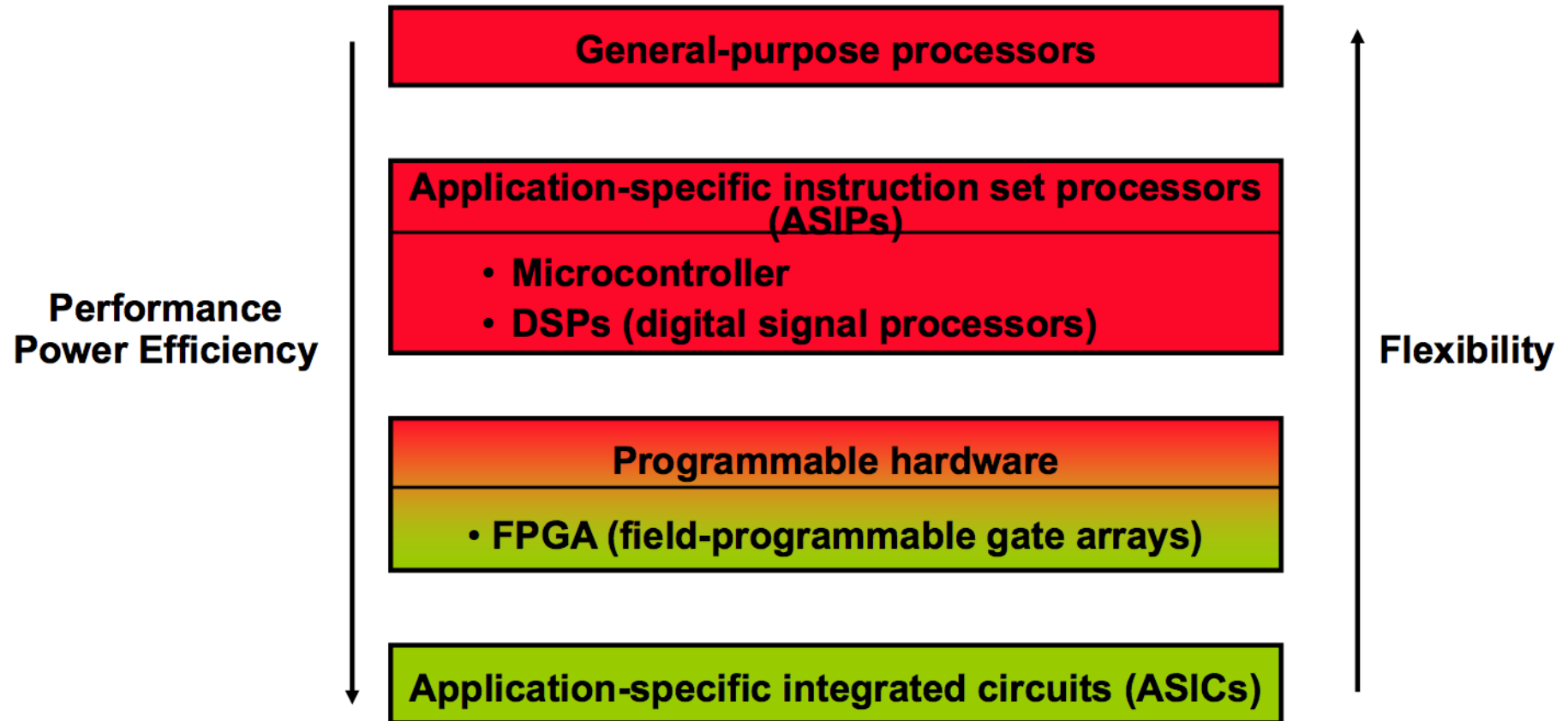
ESP32

- ESP8266 successor
- Hardware
 - Processor: Xtensa dual-core (or single-core) 32-bit LX6 microprocessor, operating at 160 or 240 MHz
 - Memory: 520 KiB RAM, 448 KiB ROM
 - 34 GPIO pins
 - SPI, I²C, I²S + UART
 - 12-bit ADC
 - IEEE 802.11 b/g/n Wi-Fi
 - ...
- Different kind of SDK and firmware
 - GCC toolchain, Wiring / Arduino, NodeMCU Lua-based, MicroPython Python based, ESP-Open-RTOS Open source FreeRTOS-based...



ESP32 SoC

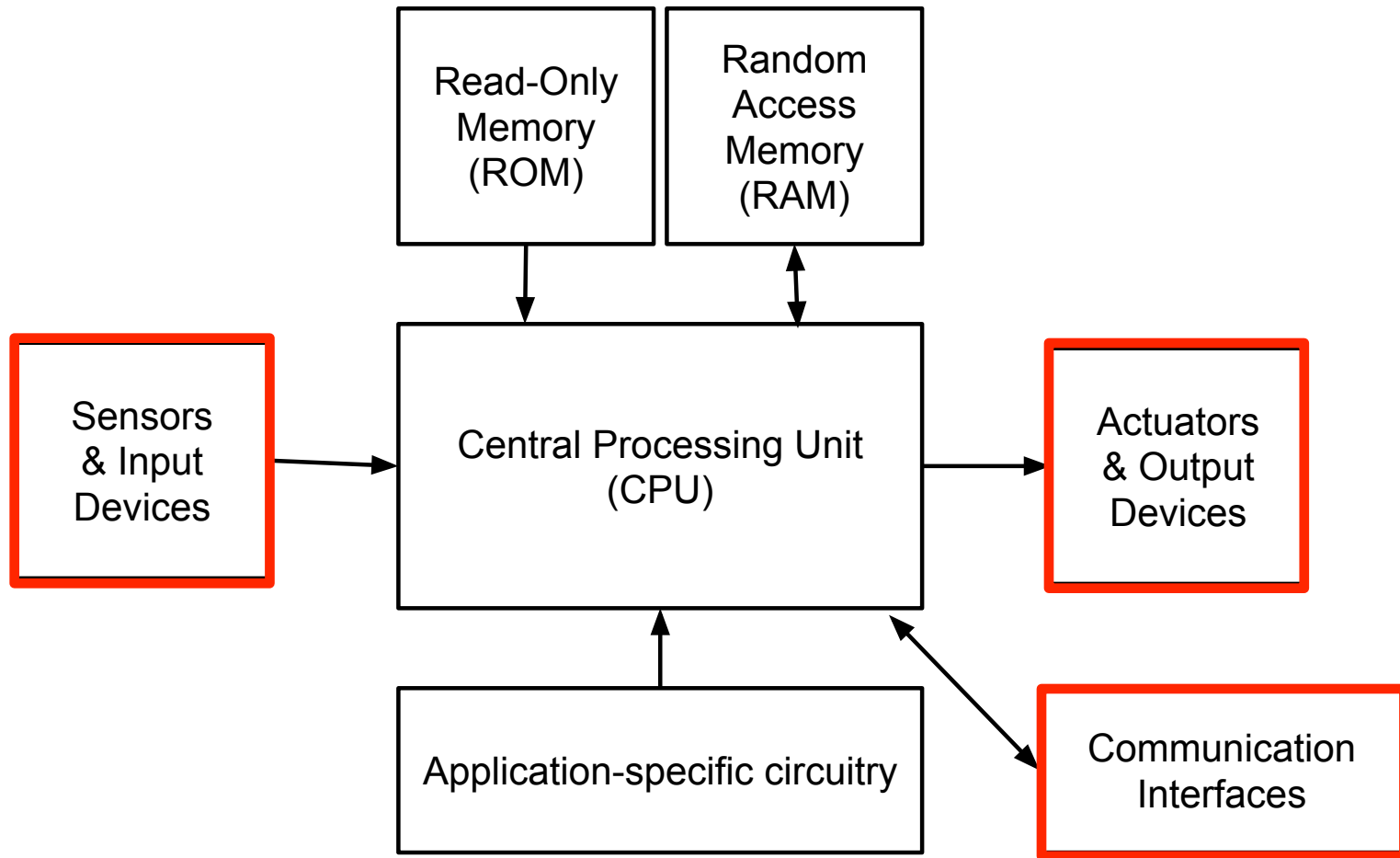
WHICH TECHNOLOGY TO CHOOSE



TREND: MCU AND SOC

- The choice depends on the requirements of the project and it is always a trade-off
- Nevertheless, solutions based on microcontrollers and SoC — and related single boards — are more and more adopted and pervasive
 - they ease the development of families of products, that can be designed to provide different kind of functionalities, ad different costs
 - flexibility, efficiency, more scalable with the complexity
 - the use of processors with a predefined instruction set often leads to a quicker implementation (than the use of custom logics)
 - this is the effect of the continuous progress of the CPU
- **Microcontrollers and SoC programmability brings effective benefits to the overall design and development process**

HARDWARE ARCHITECTURE: SENSORS, ACTUATORS, BUS



SENSORS AND ACTUATORS

- An embedded system interacts with the environment where it is situated by means of sensors and actuators
- **Sensors**
 - *transducers* devices that make it possible to *measure* a certain specific physical phenomenon (e.g. temperature, humidity, acceleration, distance, etc) or detect and quantify a chemical concentration (e.g. smoke)
 - provide a measurable representation of a phenomenon in a specific scale or interval
 - can be either **analog** or **digital**
 - *analog* => the electrical measurement produced in output — either voltage or current — changes according to variations of the measured phenomenon in input
 - for analog sensor, the microcontroller integrates a analog-digital-converter (ADC)
 - *digital* => the electrical value is mapped into a discrete set of values (e.g. two values, HIGH and LOW)
- **Actuators**
 - devices that produce some measurable effect on the environment

SENSOR EXAMPLES



Proximity sensor



Temperature sensor



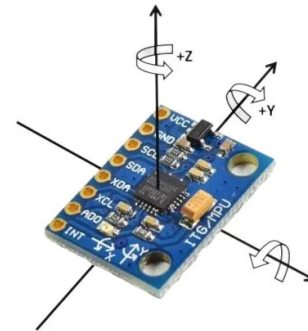
GPS



Luminosity sensor



Presence sensor (PIR)



MPU-6050
3 axes Gyro+Accelerometer

BUS AND COMMUNICATION PROTOCOLS

- The interaction between the MCU/microprocessor and sensors/actuators is enabled by communication channels (bus) based on specific communication protocols
- In embedded systems buses/protocols are typically **serial**
 - i.e. multi-bits words are sent/receive as a sequential stream of bits
- Examples
 - **I²C**, **SPI**, JTAG
 - CAN-bus

ASYNCHRONOUS SERIAL - UART

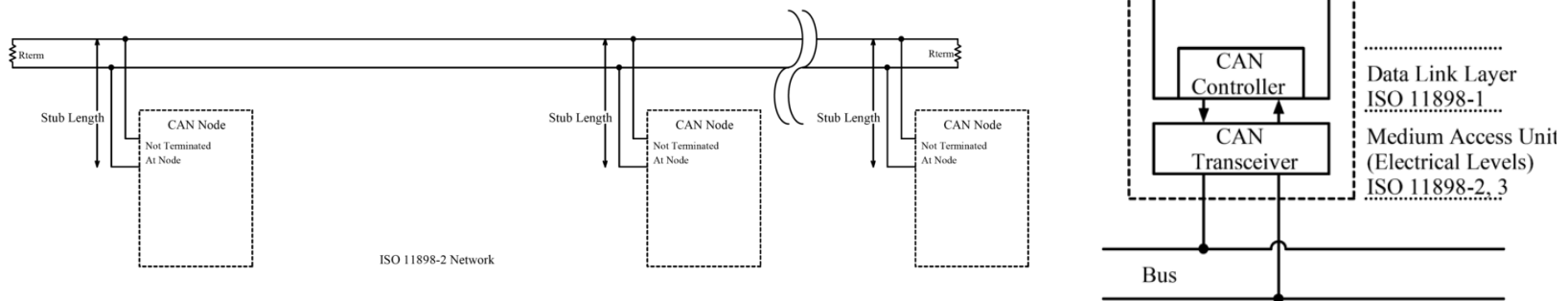
- Universal Asynchronous Receiver-Transmitter
 - oldest serial protocol, supported by any MCU
 - allows for converting parallel bit streams into a sequential asynchronous stream, and viceversa
 - actually UART does not generate or receive directly the signals to be converted: this is typically done by separate interface devices/protocols such as RS-232, Bluetooth, USB,...
- USART (Universal Synchronous/Asynchronous Receiver-Transmitter) extends the protocol with a line for a clock signal for synchronisation

I²C, SPI e JTAG

- **I²C (Inter-Integrated Circuit)**
 - widespread serial bus/protocol *synchronous*
 - very simple: only 2 lines — 1 for a clock, 1 for data
- **SPI (Serial Peripheral Interface)**
 - another synchronous bus/protocol
 - more complex than I²C
- **JTAG**
 - standard protocol for testing devices and useful for debugging embedded software

CAN-BUS

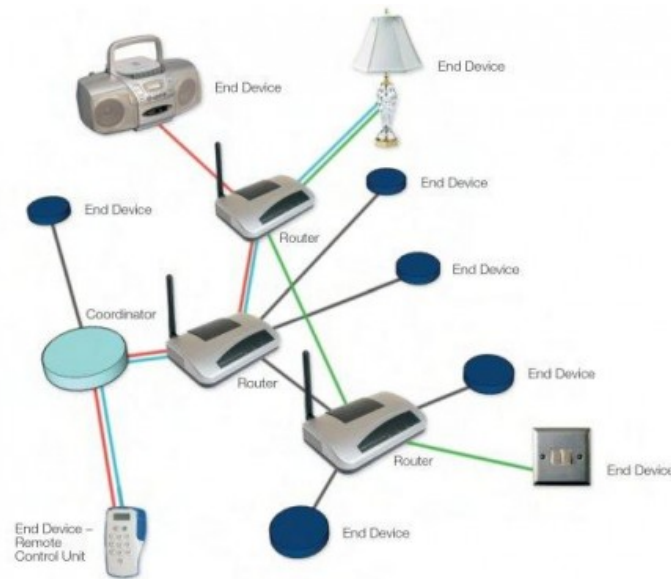
- Controller Area Network (CAN-bus)
 - standard serial protocol used in **automotive**, introduced by Bosch
 - designed to connect multiple control units
 - robust enough to work in environments with electromagnetic interferences
 - based on message multicast



- A node can be also a sensor/actuator + host processor

COMMUNICATION INTERFACE AND TECHNOLOGIES

- Technologies and standards that allow for a wired and wireless communication with other systems (either embedded or not)
- **Wireless** in particular
 - **Bluetooth** and Bluetooth Low-Energy (**BLE**)
 - ZigBee
 - Z-Wave
 - LoRoWan
 - Wifi
 - ...

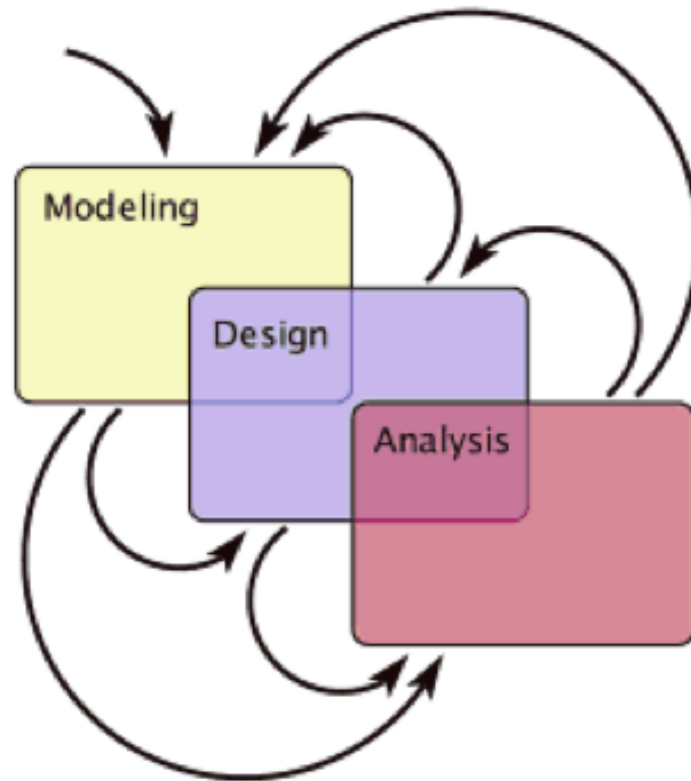


<https://it.emcelettronica.com/zigbee-come-funziona>

EMBEDDED SYSTEM DEVELOPMENT PROCESS: AN OVERVIEW

EMBEDDED SYSTEM ENGINEERING

- Iterative process
 - modelling
 - design
 - analysi



(IES book, p. 8)

EMBEDDED SYSTEM ENGINEERING: THE PROCESS

- **Modeling**
 - process to get a deep understanding and knowledge about the system to be built
 - such knowledge is represented in terms of *models*, which are the result of this process
 - they represent *what* the system does
- **Design**
 - process targeted to create artefacts that represent the system
 - it specifies *how* the system does what it does
 - modelling can have an important role also in this stage
 - providing an abstract description about how the the system should work, in spite of any specific implementation
- **Analysis**
 - process targeted to get a deep understanding about how the system behaves
 - it specifies why the system does what it does (or it does not do what it was supposed to do)

MODELLING

- **Model** = rigorous description of the relevant aspects of a system, useful to explicitly specify and understand properties of the system and its dynamics
 - dynamics = temporal evolution
- Embedded systems and CPS are systems composed by physical subsystems integrated with computational and network systems => a model of an embedded system can include all three parts (physical, computing, networking)
 - modelling the physical part
 - modelling the logical/computational part
 - software, algorithms

MODELLING THE DYNAMIC BEHAVIOUR

- Modelling the physical part => continuous-time dynamics
 - mathematical models & control theory
 - typically studied in automation & control engineering courses
- **Modelling the logical part** => discrete-time dynamics
 - **state machines** and their composition
 - **tasks** & concurrent models
 - this course

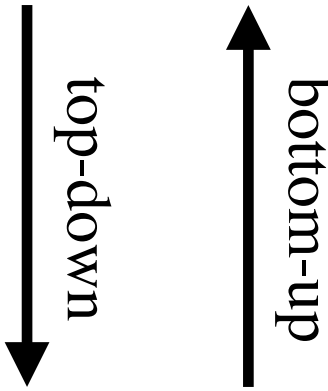
DESIGN AND DEVELOPMENT

- Embedded system design:
 - choose the proper HW architecture and technology
 - +
 - choose the proper SW architecture

ANALYSIS/VERIFICATION

- Every system is designed to satisfy some specific **requirements**
 - this is true in particular for *safety-critical systems*
- Requirements should be expressed rigorously in terms of properties or specifications, in some formal language
 - “*a design without specification cannot be right or wrong, it can be only surprising!*” (Young et al., 1985)
- Tools
 - formal languages to specify properties
 - e.g. temporal logics
 - techniques for analysing the properties and to verify them upon the model describing the dynamics of the system
 - e.g. model-checking

TOP-DOWN AND BOTTOM-UP DESIGN

- Design can be either top-down or bottom-up, through different levels
 - **requirements**
 - **specification**
 - **architecture**
 - **components**
 - **system integration**
- 

REQUIREMENTS

- Requirements elicitation and collection from customers and domain experts about what the system is expected to do
 - **functional** requirements
 - functions/tasks that the system must perform
 - **non-functional** requirements
 - performance, cost (manufacturing and NRE), physical dimension, weight, power consumption, ..
- Use of *requirement forms*
 - including name, objective, inputs, outputs, functions, performance, cost, power consumption, size

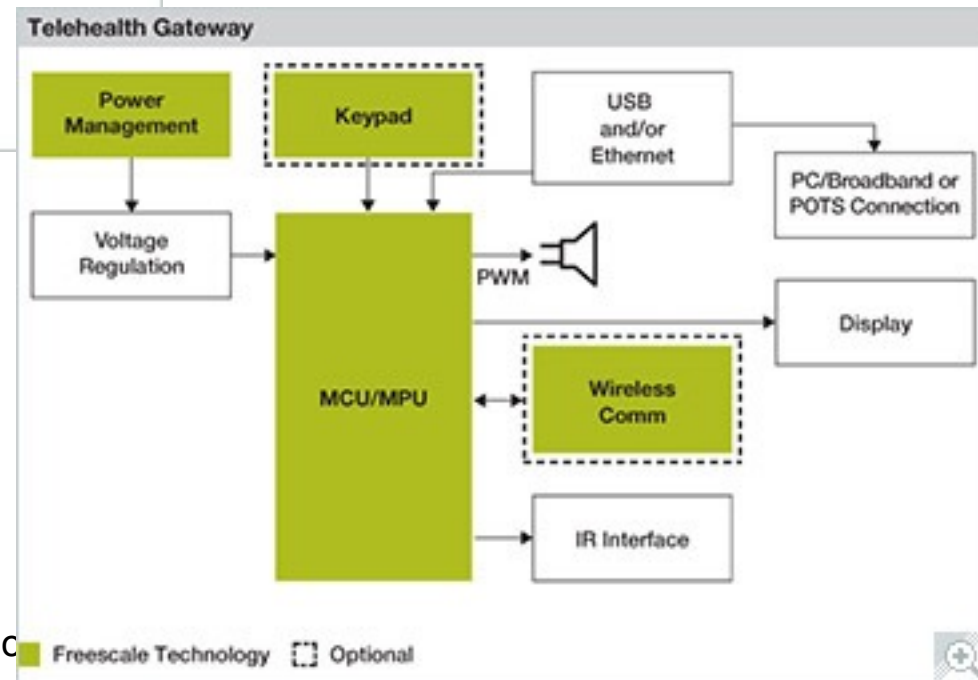
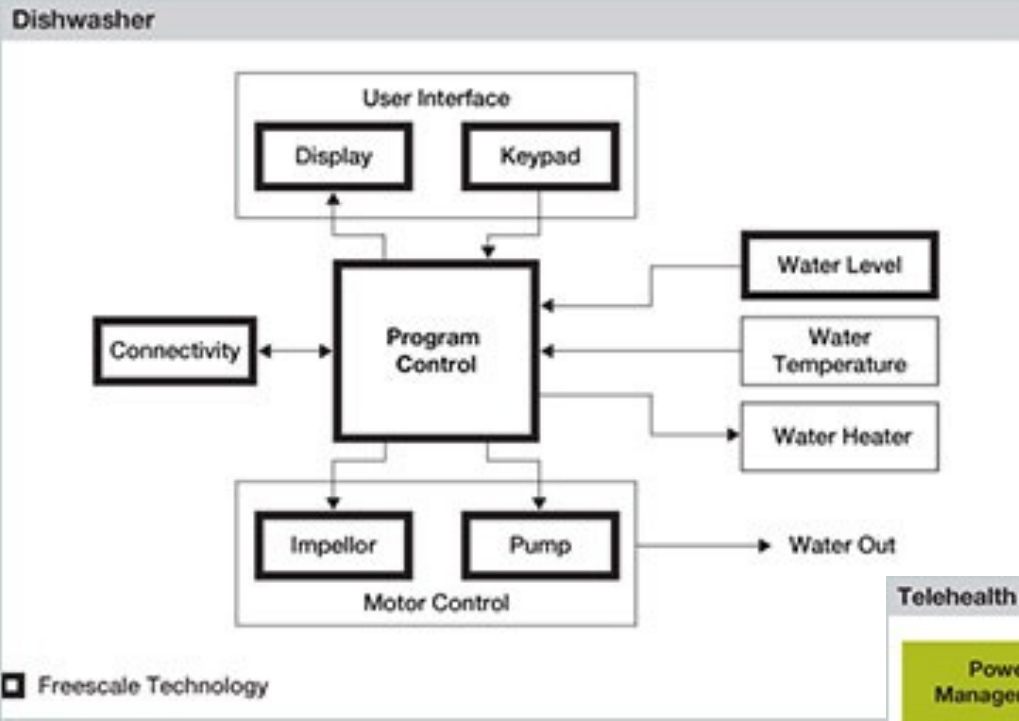
SPECIFICATION

- Rigorous description of requirements
 - it is a contract between the customer and designers
- It should be clear and detailed enough to allow for verifying customer requirements and expectation
 - not ambiguous
- **UML language**, formal languages
 - e.g. temporal logics

DESIGN & ARCHITECTURE

- Description of how the system implements the functions described in specs
- **Architecture**
 - plan of the overall structure of the system, to be used to drive the development of components
 - it describes which components are part of the system and how they interact
- **Block Diagram**
 - hardware and software

BLOCK DIAGRAMS



FROM COMPONENT DESIGN TO SYSTEM INTEGRATION

- Individual component design and development
 - both HW and SW parts
 - e.g.: FPGA, boards, SW modules
 - some components can be ready-made, i.e. already available
 - CPU, GPS receivers, topographic databases, services..
- System integration
 - integration of components as defined by the architecture
 - critical part of the design

PROGRAMMING, DEVELOPMENT AND DEBUGGING

- Embedded software development
 - traditional or *model-driven*
- Typically on host computers
 - tool-chain to create executables, to be uploaded and executed on the embedded system
- High-level languages and platforms
 - C/C++ languages and system libraries
 - pervasively used in microcontroller-based solutions
 - in embedded systems with an OS => widespread use of languages/platforms with a runtime architecture
 - languages: C/C++ but also JavaScript, Java, C#, Python, Rust, ...
 - platforms: Embedded Java, Android,...
- Debugging
 - exploiting serial connection for logging messages on a host PC
 - JTAG interface/protocol

REFERENCES

- Wayne Wolf. *Computers as Components - Principles of Embedded Computer System Design*. Morgan Kaufman
 - chapter 1
- [IES Book] Ed Lee and S. Seshia. *Introduction to Embedded Systems. A Cyber-Physical Systems Approach*. UC Berkley
 - chapter 1
- Frank Vahid and Tony Givargis. *Embedded System Design - A unified Hardware/Software perspective - Introduction*. Wiley
 - chapter 1